Assignment 1 Report: sdd-flashcards

12 October 2026 · Dominic Potec

1. Introduction

My app is a spaced repetition flashcard app in the style of Anki, built as one Flask process over one SQLite file. Repository: github.com/dom-potec/sdd-flashcards.

A user creates decks of cards, with questions on the front and answers on the back, and studies the cards that are due today, grades each answer from 0 to 5, and the app schedules the next review with the SM-2 algorithm.

The backend has two feature domains, each its own Python package with its own tables:

- **Content** (`app/content`): decks and cards, full create/list/view/delete. Owns `decks` and `cards`.
- **Scheduling** (`app/scheduling`): when each card is next due, computed from review grades. Owns `card_schedule` and `reviews`.

Stack: Python 3.12, Flask 3.1 with Jinja templates, the standard-library `sqlite3` module, pytest with pytest-cov. Three third-party packages, 34 files, no JavaScript. The app runs with `python app.py` on a laptop and inside the Docker container.

2. SDLC model: incremental, one increment per day

I used an incremental model: six build days, each ending in a pushed, working increment, and from day 3 a green test suite. Each day had a scope of around three commits, planned up front in a build plan, so that the commit history and the ADR log would record decisions as they were made rather than at the end.

SMART goals

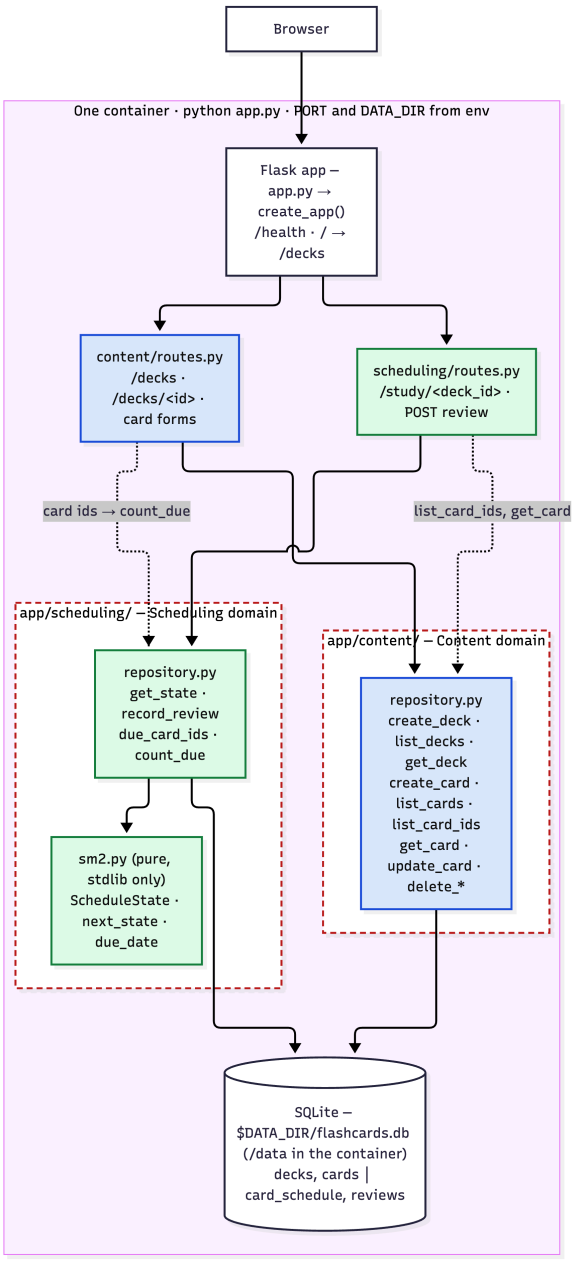
- Specific: two backend domains, Content and Scheduling, in one Flask process over one SQLite file, runnable locally and in the provided container.
- Measurable: at least 70% test coverage on `app/`, at least 12 commits over 6 days, 5 ADRs on at least 3 dates, a passing `run.sh`.
- Achievable: three dependencies, about 30 files, no authentication, no JavaScript.
- Relevant: the two domains are separate packages with a documented seam.
- Time-bound: last code push Friday 9 October; report submitted by Monday 12 October.

How it went in practice

Day	Planned	Actual	Increment
1	Thu 1 Oct	Thu 1 Oct	App factory, config, ADR-1, ADR-2, AI log
2	Fri 2 Oct	Mon 5 Oct	Schema, `db.py`, content repository, ADR-3
3	Sat 3 Oct	Tue 6 Oct	SM-2 pure functions, their tests, scheduling repository
4	Sun 4 Oct	Wed 7 Oct	Deck and card pages, study flow, ADR-4, smoke tests
5	Mon 5 Oct	Thu 8 Oct	Repository tests, coverage config, ADR-5
6	Tue 6 Oct	Fri 9 Oct	Dockerfile, container check, README, diagrams

Two things changed in practice. First, I delayed everything by 3 days starting from day 2, but stuck to the plan instead of catching up in one day, to be compliant with the commit rules. I also saw the updated instructions in the repo on 7 October, so I had to adjust the plan to account for the Dockerfile and the passing contract checker: `pyproject.toml` had to be removed for the checker, and the dependency versions also had to be pinned. In the end I have 21 commits over 7 days.

3. Architecture



Architecture. Dashed red boxes mark the seam between the two domains; dotted arrows are the only cross-domain calls.

The whole app is one Flask process in one container. The two domains are two separate Python packages, `app/content/` and `app/scheduling/`. Each package has its own repository module that talks to its own tables, and neither package imports the other. The only place the two meet is the `routes.py` files, which handle the web requests and call whichever repository they need. In the diagram, the red boxes mark that separation. If the app were ever split into two services, the cut would

go between the two boxes, and the two dotted arrows are the only calls that would turn into HTTP requests.

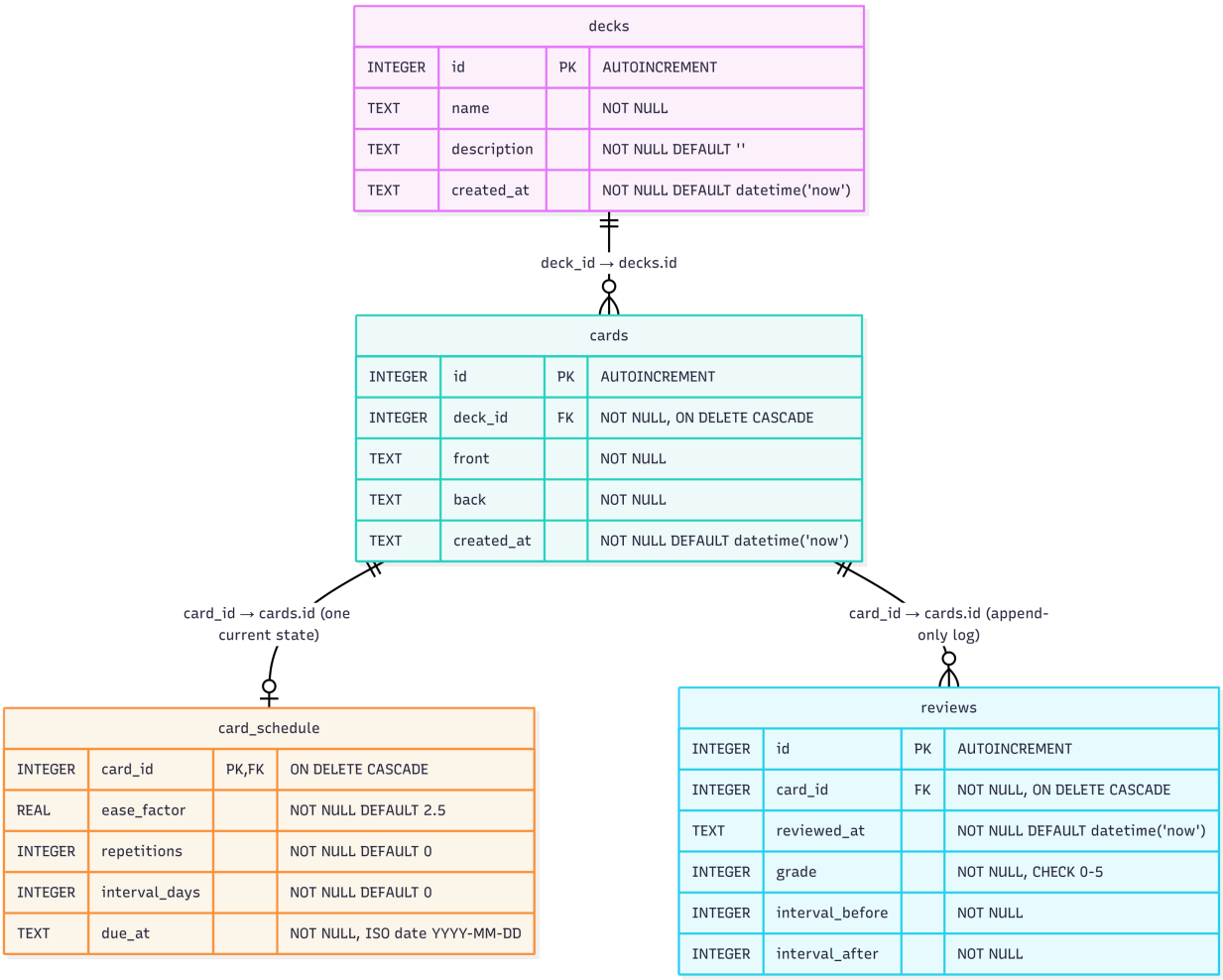
What crosses the seam is card ids and nothing else:

- `content/routes.py` passes a deck's card ids to `scheduling.repository.count_due` to show the "due today" column.
- `scheduling/routes.py` asks `content.repository.list_card_ids` for a deck's ids, asks `scheduling.repository.due_card_ids` which of those are due, then asks `content.repository.get_card` for the text of the first one.

Scheduling never sees card text. Content never sees ease factors or due dates. `sm2.py` imports only the standard library, so the SM-2 maths can be tested with no database at all. Each request opens one SQLite connection in `g` and closes it on teardown. The whole `app/` package is about 300 lines of Python, where every function fits in two sentences.

4. Data model

Four tables in one SQLite file, created by `app/schema.sql` on every start with `CREATE TABLE IF NOT EXISTS`. Content owns `decks` and `cards`. Scheduling owns `card_schedule` and `reviews`. The only link between the domains is the foreign key from the two scheduling tables to `cards.id`, declared `ON DELETE CASCADE` so that deleting a card or a deck also removes its schedule and history (the `PRAGMA foreign_keys = ON` in `get_conn` is what makes SQLite enforce it).



Database schema. `decks` and `cards` belong to Content; `card_schedule` and `reviews` to Scheduling.

A card with no `card_schedule` row has never been reviewed and is treated as due now. `reviews` is write-only in this assignment. It exists so that statistics can be added later without changing the schema. The diagram matches `schema.sql` and ADR-3 column for column; the Mermaid source is kept in `docs` beside the PNG.

5. Design decisions

The full records are in `ADR.md`. They were written on four dates (1, 5, 7 and 8 October) as each decision was made.

#	Decision	Why, and what was rejected
1	**Flask** with Jinja and the standard-library `sqlite3` (1 Oct)	The app needs routing, templates and forms, nothing more. Django was rejected for its ORM, admin and auth that this app would not use; FastAPI because async I/O buys nothing for synchronous `sqlite3` calls and it still needs a template layer.
2	**Two sibling packages with a one-way seam** (1 Oct)	`content/` and `scheduling/` never import each other's domain modules; only `routes.py` calls both, and Scheduling receives card ids only. A single models module was rejected: quicker, but SM-2 could not be tested without the rest of the app.
3	**Scheduling state in `card_schedule`, history in `reviews`** (5 Oct)	One current-state row per card plus an append-only log, both keyed on `cards.id` with cascade. Putting `ease_factor` and `due_at` on `cards` was rejected because Content would then own Scheduling's data.
4	**Test the scheduler and repositories first, routes only by smoke tests** (7 Oct)	`sm2.py` and both repositories are tested in detail against a temporary database; the routes get four smoke tests. Route-by-route test-client tests were rejected: they inflate coverage without showing the scheduler computes the right interval.
5	**No user accounts** (8 Oct)	Single-user by design. Flask-Login with a `users` table was rejected: one more dependency and a table touching both domains, when neither domain needs to know who owns a deck. Adding `owner_id` to `decks` later is one migration that does not touch Scheduling.

6. Testing

39 tests pass and coverage of `app/` is 83% with branch coverage on, measured with pytest-cov:

```
pytest --cov=app --cov-report=term-missing
```

Module	Statements	Coverage
`scheduling/sm2.py`	24	100%
`content/repository.py`	45	100%
`scheduling/repository.py`	27	100%
`db.py`, `config.py`, `__init__.py`	43	97–100%
`content/routes.py`, `scheduling/routes.py`	85	55–58%

The priority follows ADR-4: the core logic (the SM-2 maths and the SQL in both repositories) is at 100%, and the thin route handlers are left to smoke tests. The uncovered 35 statements are all route

bodies. Every repository test runs against a real SQLite file created by the `conn` fixture, so the foreign-key cascade and the `IF NOT EXISTS` schema are tested properly.

7. Setup and container contract

The README covers both ways to run the app. Locally:

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python app.py            # http://localhost:8000
```

In Docker, with the course Dockerfile template:

```
docker build -t sdd-flashcards .
docker run -p 8000:8000 -v sdd-flashcards-data:/data sdd-flashcards
```

Configuration is entirely by environment variable, each with a default: `PORT` (8000), `DATA_DIR` (`./data`, `/data` in the container), `DB_PATH` (`$DATA_DIR/flashcards.db`), `SECRET_KEY` (a dev default). The app binds `0.0.0.0`, creates its schema on first boot against an empty data directory, ships no seed data and never drops a table, so a restart changes nothing.

The course contract checker `container/run.sh` passed on 9 October (I had something else running on port 8000 so I used port 8100; the second start on 9123 is the checker's own `PORT` override test):

```
PASS  one Dockerfile, one manifest (requirements.txt)
PASS  image built
PASS  image size 53 MB
PASS  HTTP 302 from http://localhost:8100/
PASS  found in /data: flashcards.db
PASS  volume at /data persists
PASS  row counts unchanged across restart: card_schedule=0 cards=0 decks=0 reviews=0
PASS  HTTP 302 from http://localhost:9123/
=== ALL CHECKS PASSED ===
```

8. AI disclosure

I acknowledge the use of Claude for planning and Claude Code for implementation, to draft the SM-2 scheduling functions, the SQLite repository modules, Flask routes, Jinja templates, the tests, Mermaid code for the diagrams and to fill in some tables of this report with information from the repo. I also checked different things with Claude and had it explain different things like `ON DELETE CASCADE` and differences between SQL and SQLite as well as Flask.

The prompts used include "start with the database shema and the init.py and continue according to block 4", "lets do the sm2 scheduling", "lets start with block 9. repository.py", "lets do the study flow, workflow according to the plan", "lets do the test for the reporisotires" and "lets also add the dockerfile update the README and make it match the template in the professors repo".

All the prompting can be seen in `AI_USAGE.md`; the outputs were drafts, which I either used or modified, which is all logged in the file. `ADR.md` entries were rewritten by me after Claude's drafts, and the "in my own words" column of `AI_USAGE.md` is me explaining what the changes consist of.